TP1

Mathieu Waharte - 11/09/2025

Advanced Networks -- APP5 -- Labs

- **TP1 et TP2 :**

https://github.com/KatharaFramework/Kathara-Labs/tree/main/

- **TP1:**

Tutorials:

Exercice1: capture-packets
Exercice2: traffic-control : fixed-delay
Exercice3: traffic-control : limit-bandwidth

Main-labs -> Basic-topics:

Exercice4: basic-IPV4

- **TP2:**

Main-labs -> Application-level:

Exercice1: web-server
Exercice2: load-balancer
Exercice3: dns

- **TP3:**

Network Tomography (je vous enverrai l'énoncé ultérieurement)

Exercice 1 - Capture Pack/Installation de Kathará

1. Télécharger et installer Docker Desktop (si pas déjà installé)
2. Télécharger et installer Kathará
3. Avec Docker Desktop en cours d'exécution, ouvrez un terminal et exécutez la commande suivante pour vérifier que Kathará est correctement installé :

   ```
   kathara --version
   ```

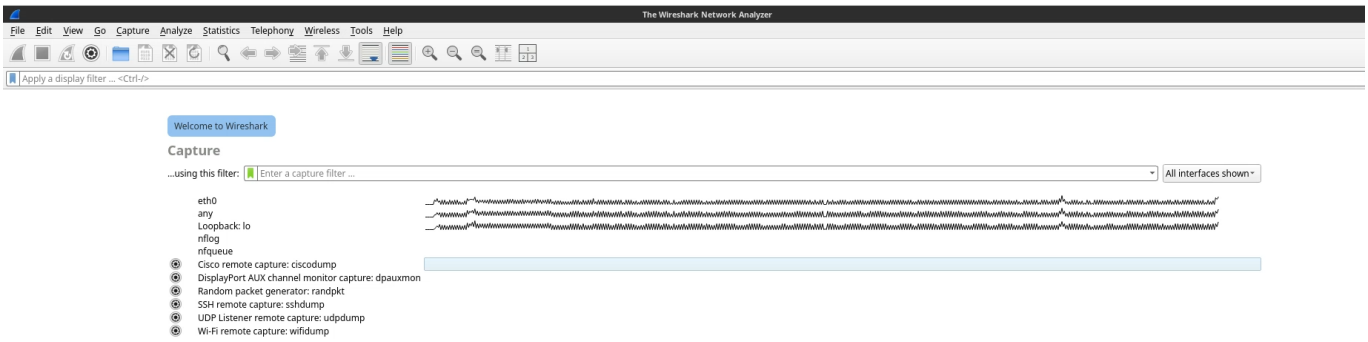
4. Allez dans votre répertoire de lab (contenant les fichiers `lab.conf`, `pc1.startup`, `pc2.startup`) et exécutez `kathara lstart` pour télécharger les images et démarrer le lab. Cela prend un certain temps la première fois. Ensuite, exécutez `kathara check` pour vérifier que tout fonctionne correctement.
5. `kathara lstop` pour arrêter le lab.
6. Ensuite, appliquez les correctifs pour le problème de redirection de port de Docker Desktop : exécutez ``kathara vstart -n pc1 --bridged --port 8080:80 --image kathara/base --exec "/etc/init.d/apache2 start"`. (Source, le port forwarding dans Kathará rencontre un problème lorsque l'interface bridgée n'est pas la première sur un appareil, c'est un problème de Docker Desktop (docker/for-mac#6978).)
7. Maintenez, pour Docker Desktop, commentez `wireshark[0]=A` dans `lab.conf`.
8. Pour ajouter l'interface A à pc1, exécutez `kathara vconfig -n pc1 --add A`.
9. Enfin, ouvrez votre navigateur à l'adresse `localhost:3000` et vous devriez voir l'interface de Wireshark. Si nécessaire, connectez-vous avec l'utilisateur `abc` et le mot de passe `abc`.

Pour ouvrir un autre lab, il suffit de se placer dans le répertoire du lab et de lancer `kathara lstart`.

Autres commandes:

- `kathara linfo` pour obtenir le statut de toutes les machines virtuelles du lab et leur liste
- `kathara vclean -n pc1` pour nettoyer la machine virtuelle pc1 (déconnecter et supprimer les fichiers temporaires)
- `kathara lclean` pour nettoyer l'ensemble du lab (déconnecter et supprimer les fichiers temporaires pour toutes les machines virtuelles)

Introduction à Kathará pour plus d'informations.



On remarque que l'installation de Kathará a fonctionné et que wireshark observe le traffic sur eth0. On

utilisera Wireshark pour les exercices suivants.

Exercice 2 - Fixed Delay

Pour cet exercice, je me suis placé dans le répertoire `fixed-delay` puis j'ai simplement lancé `kathara lstart` pour démarrer le lab. Après avoir vérifié que tout fonctionnait avec `kathara info`, je me suis placé dans la console de pc1. J'ai ensuite effectué le ping mentionné dans le README:

```
root@pc1:/# ping 200.0.0.2 -c 10
PING 200.0.0.2 (200.0.0.2) 56(84) bytes of data.
64 bytes from 200.0.0.2: icmp_seq=1 ttl=63 time=23.9 ms
64 bytes from 200.0.0.2: icmp_seq=2 ttl=63 time=11.8 ms
64 bytes from 200.0.0.2: icmp_seq=3 ttl=63 time=12.1 ms
64 bytes from 200.0.0.2: icmp_seq=4 ttl=63 time=11.5 ms
64 bytes from 200.0.0.2: icmp_seq=5 ttl=63 time=11.7 ms
64 bytes from 200.0.0.2: icmp_seq=6 ttl=63 time=11.9 ms
64 bytes from 200.0.0.2: icmp_seq=7 ttl=63 time=12.1 ms
64 bytes from 200.0.0.2: icmp_seq=8 ttl=63 time=11.5 ms
64 bytes from 200.0.0.2: icmp_seq=9 ttl=63 time=12.7 ms
64 bytes from 200.0.0.2: icmp_seq=10 ttl=63 time=11.5 ms

--- 200.0.0.2 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9018ms
rtt min/avg/max/mdev = 11.473/13.064/23.937/3.640 ms
root@pc1:/#
```

On remarque que le ping fonctionne et que le délai est bien d'environ 10ms (à part un "cold start"), comme prévu.
Le routeur r1 est bien connecté aux deux domaines et les commandes de routing fonctionnent correctement.

Exercice 3 - Limit Bandwidth

Pour cet exercice, je me suis placé dans le répertoire `fixed-delay` puis j'ai simplement lancé `kathara lstart` pour démarrer le lab. Après avoir vérifié que tout fonctionnait avec `kathara info`.

Sur pc2, j'ai lancé la commande `iperf3 -s` pour lancer le serveur de test de débit.
Sur pc1, j'ai lancé la commande `iperf3 -c 200.0.0.2` pour lancer le client de test de débit.
Voici le résultat:

- Sur pc2 (serveur):

```
root@pc2:/# iperf3 -s
-----------------------------------------------------------
Server listening on 5201 (test #1)
-----------------------------------------------------------
Accepted connection from 100.0.0.2, port 42270
[  5] local 200.0.0.2 port 5201 connected to 100.0.0.2 port 42280
[ ID] Interval           Transfer     Bitrate
[  5]   0.00-1.00   sec  31.1 KBytes   255 Kbits/sec
[  5]   1.00-2.00   sec  28.3 KBytes   232 Kbits/sec
[  5]   2.00-3.00   sec  29.7 KBytes   243 Kbits/sec
[  5]   3.00-4.00   sec  29.7 KBytes   243 Kbits/sec
[  5]   4.00-5.00   sec  29.7 KBytes   243 Kbits/sec
[  5]   5.00-6.00   sec  29.7 KBytes   243 Kbits/sec
[  5]   6.00-7.00   sec  32.5 KBytes   266 Kbits/sec
[  5]   7.00-8.00   sec  29.7 KBytes   243 Kbits/sec
[  5]   8.00-9.00   sec  29.7 KBytes   243 Kbits/sec
[  5]   9.00-10.00  sec  29.7 KBytes   243 Kbits/sec
[  5]  10.00-10.03  sec  1.41 KBytes   404 Kbits/sec
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate
[  5]   0.00-10.03  sec   301 KBytes   246 Kbits/sec                  receiver
-----------------------------------------------------------
Server listening on 5201 (test #2)
-----------------------------------------------------------
```

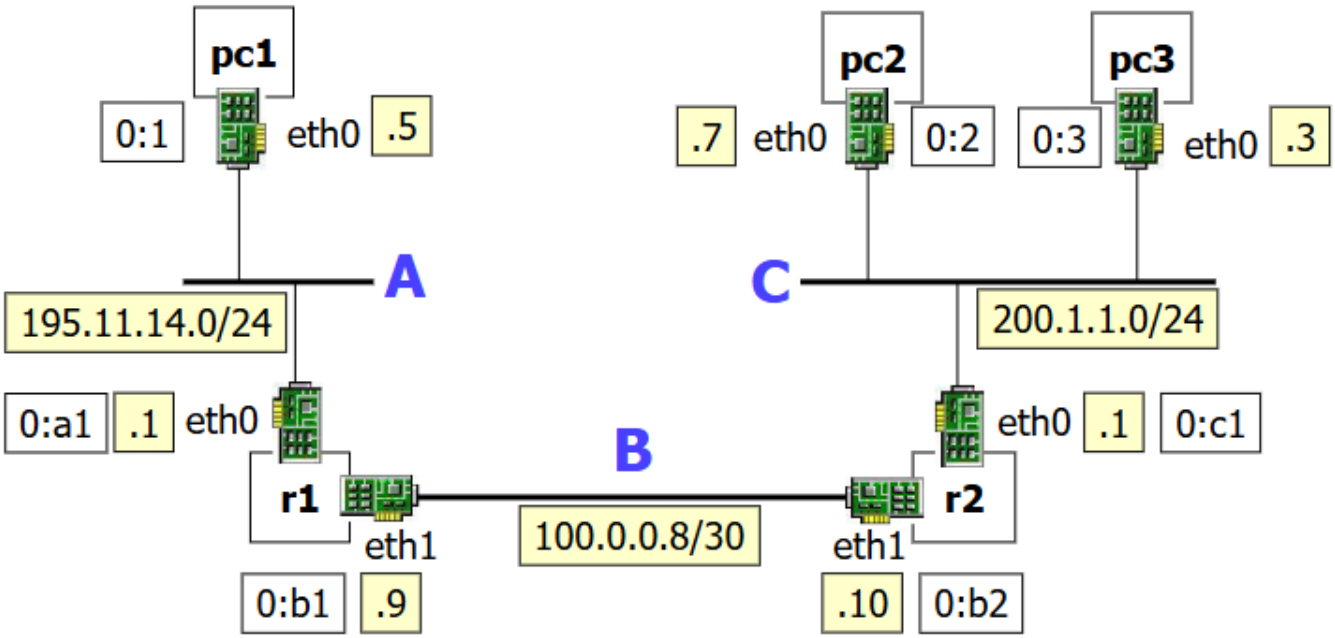
- Sur pc1 (client):

```
root@pc1:/# iperf3 -c 200.0.0.2
Connecting to host 200.0.0.2, port 5201
[  5] local 100.0.0.2 port 42280 connected to 200.0.0.2 port 5201
[ ID] Interval           Transfer     Bitrate         Retr  Cwnd
[  5]   0.00-1.00   sec   107 KBytes   879 Kbits/sec   23   1.41 KBytes
[  5]   1.00-2.00   sec  48.1 KBytes   394 Kbits/sec    7   1.41 KBytes
[  5]   2.00-3.00   sec  0.00 Bytes  0.00 bits/sec    7   1.41 KBytes
[  5]   3.00-4.00   sec  39.6 KBytes   325 Kbits/sec    7   1.41 KBytes
[  5]   4.00-5.00   sec  39.6 KBytes   324 Kbits/sec    7   1.41 KBytes
[  5]   5.00-6.00   sec  0.00 Bytes  0.00 bits/sec    7   1.41 KBytes
[  5]   6.00-7.00   sec  39.6 KBytes   324 Kbits/sec    7   2.83 KBytes
[  5]   7.00-8.00   sec  39.6 KBytes   325 Kbits/sec    7   2.83 KBytes
[  5]   8.00-9.00   sec  39.6 KBytes   324 Kbits/sec    7   2.83 KBytes
[  5]   9.00-10.00  sec  0.00 Bytes  0.00 bits/sec    7   2.83 KBytes
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate         Retr
[  5]   0.00-10.00  sec   354 KBytes   290 Kbits/sec   86             sender
[  5]   0.00-10.03  sec   301 KBytes   246 Kbits/sec                  receiver

iperf Done.
```

On remarque que le débit est bien limité à environ 256kbps, comme prévu. L'envoie sur pc1 n'est pas limité, mais le débit reçu sur pc2 l'est bien. Le dernier paquet est reçu avec un débit plus élevé car le test s'arrête et le buffer se vide.

Exercice 4 - IPv4



De l'énoncé, j'ai retenu les étapes suivantes pour cet exercice:

1. Sur pc1, pc2, pc3, r1 et r2, exécutez la commande `ip address` pour vérifier les adresses IPv4 assignées aux interfaces et examinez les interfaces eth et loopback.

2. Sur pc1, pc2 et pc3 : exécutez la commande `routel` pour vérifier la présence d'une route par défaut, le préfixe de loopback.

3. Sur r1 et r2 : exécutez la commande `routel` pour vérifier la table de routage, trouver les LAN A, B, C et r1.

4. Sur pc3 :

   - inspectez le cache ARP : `arp -n` (initialement vide, le ping nécessite une résolution d'adresse qui sera stockée dans le cache ARP)
   - exécutez une commande ping vers pc2 : `ping 200.1.1.7`
   - inspectez à nouveau le cache ARP : `arp -n`
   - jetez un œil aux paquets capturés par Wireshark avec `kathara lconfig -n wireshark --add C` et sniff eth1.

   Notez que le trafic dans le même réseau ne passe pas par le routeur r2.

   Sur pc2 : `arp -n`. Sniff le trafic:

   - connectez le dispositif Wireshark au domaine de collision C : `kathara lconfig -n wireshark --add C`
   - ouvrez un navigateur sur la machine hôte à l'adresse `localhost:3000`
   - sniff eth1 Trouvez les requêtes ARP sur Wireshark.

5. PC1-PC2 ping:

- Connectez le dispositif Wireshark au domaine de collision B avec `kathara lconfig -n wireshark --add B` et sniff eth2.
- Sur pc2, pingez pc1 : `ping 195.11.14.5`
- Inspectez le cache ARP sur pc2 avant et après le ping : `arp -n`. L'adresse MAC de eth0 sur r2 devrait apparaître dans le cache ARP de pc2 après le ping, car le trafic IP est adressé en dehors du réseau local.
- Sur les routeurs, le cache ARP peut également être inspecté : `arp -n`. Ils effectuent une requête ARP chaque fois qu'ils doivent envoyer des paquets IP sur un réseau Ethernet.
- Regardez les paquets capturés par Wireshark : requêtes et réponses ARP, requêtes et réponses ICMP echo.

6. Traceroute from pc2 to pc1:

- Connectez le dispositif Wireshark au domaine de collision C avec `kathara lconfig -n wireshark --add C` et sniff eth1.
- Sur pc2, exécutez une commande traceroute vers pc1 : `traceroute 195.11.14.5 -z 1`. Vous verrez l'eth0 de r2, puis l'eth1 de r1, puis l'eth0 de pc1.
- Sur Wireshark, recherchez :
  - les paquets UDP envoyés par pc2 vers pc1 avec des valeurs TTL croissantes
  - les messages ICMP Time-to-live exceeded envoyés par r2 et r1 vers pc2
  - le dernier paquet UDP envoyé par pc2 vers pc1 avec TTL=3 et la réponse ICMP echo correspondante envoyée par pc1 vers pc2
  - les requêtes ARP unicast émises par r2 et r1 pendant le dialogue

7. Exercices:

- Vérifiez les différents messages d'erreur obtenus en essayant de pinger une destination injoignable dans le cas de :
  - destination locale
  - destination non locale
- Quels paquets sont échangés dans le domaine de collision local dans les deux cas ?

Nous allons donc suivre ces étapes:

1. Sur pc1, pc2, pc3, r1 et r2, j'ai effectué la commande `ip address` pour vérifier les adresses IPv4 assignées aux interfaces et regarder les interfaces eth et loopback.

   - Sur pc1:

```
root@pc1:/# ip address
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
38: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 00:00:00:00:00:01 brd ff:ff:ff:ff:ff:ff
    inet 195.11.14.5/24 scope global eth0
       valid_lft forever preferred_lft forever
root@pc1:/#
```

   L'adresse de eth0 est 195.11.14.5/24 et celle de loopback est 127.0.0.1/8.

- Sur pc2:

```
root@pc2:/# ip address
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
36: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 00:00:00:00:00:02 brd ff:ff:ff:ff:ff:ff
    inet 200.1.1.7/24 scope global eth0
       valid_lft forever preferred_lft forever
root@pc2:/#
```

  L'adresse de eth0 est 200.1.1.7/24 et celle de loopback est 127.0.0.1/8.

- Sur pc3:

```
root@pc3:/# ip address
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
37: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 00:00:00:00:00:03 brd ff:ff:ff:ff:ff:ff
    inet 200.1.1.3/24 scope global eth0
       valid_lft forever preferred_lft forever
root@pc3:/#
```

  L'adresse de eth0 est 200.1.1.3/24 et celle de loopback est 127.0.0.1/8.

- Sur r1:

```
root@r1:/# ip address
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
35: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 00:00:00:00:00:a1 brd ff:ff:ff:ff:ff:ff
    inet 195.11.14.1/24 scope global eth0
       valid_lft forever preferred_lft forever
40: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 00:00:00:00:00:b1 brd ff:ff:ff:ff:ff:ff
    inet 100.0.0.9/30 scope global eth1
       valid_lft forever preferred_lft forever
```

  On a 2 interfaces ethernet. L'adresse de eth0 est 195.11.14.1/24 et celle de eth1 est 100.0.0.9/30.

- Sur r2:

```
root@r2:/# ip address
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
39: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 00:00:00:00:00:c1 brd ff:ff:ff:ff:ff:ff
    inet 200.1.1.1/24 scope global eth0
       valid_lft forever preferred_lft forever
41: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 00:00:00:00:00:b2 brd ff:ff:ff:ff:ff:ff
    inet 100.0.0.10/30 scope global eth1
       valid_lft forever preferred_lft forever
```

  On a 2 interfaces ethernet. L'adresse de eth0 est 200.1.1.1/24 et celle de eth1 est 100.0.0.10/30.
  On remarque que les adresses IP sont bien assignées comme prévu dans l'énoncé.

2. Sur pc1, pc2 et pc3, j'ai effectué la commande `routel` pour vérifier la présence d'une route par défaut et la préfixe de loopback.

- Sur pc1:

```
root@pc1:/# routel
Dst             Gateway         Prefsrc         Protocol Scope   Dev             Table
default         195.11.14.1                                      eth0
195.11.14.0/24                  195.11.14.5     kernel   link    eth0
127.0.0.0/8                     127.0.0.1       kernel   host    lo              local
127.0.0.1                       127.0.0.1       kernel   host    lo              local
127.255.255.255                 127.0.0.1       kernel   link    lo              local
195.11.14.5                     195.11.14.5     kernel   host    eth0            local
195.11.14.255                   195.11.14.5     kernel   link    eth0            local
root@pc1:/#
```

  On remarque que la routeur est r1 (195.11.14.1) qui est bien le routeur présent sur le réseau A.

- Sur pc2:

```
root@pc2:/# routel
Dst             Gateway         Prefsrc         Protocol Scope   Dev             Table
default         200.1.1.1                                        eth0
200.1.1.0/24                    200.1.1.7       kernel   link    eth0
127.0.0.0/8                     127.0.0.1       kernel   host    lo              local
127.0.0.1                       127.0.0.1       kernel   host    lo              local
127.255.255.255                 127.0.0.1       kernel   link    lo              local
200.1.1.7                       200.1.1.7       kernel   host    eth0            local
200.1.1.255                     200.1.1.7       kernel   link    eth0            local
root@pc2:/#
```

  On remarque que le routeur est r2 (200.1.1.1) qui est bien le routeur présent sur le réseau C.

- Sur pc3:

```
root@pc3:/# routel
Dst             Gateway         Prefsrc         Protocol Scope   Dev             Table
default         200.1.1.1                                        eth0
200.1.1.0/24                    200.1.1.3       kernel   link    eth0
127.0.0.0/8                     127.0.0.1       kernel   host    lo              local
127.0.0.1                       127.0.0.1       kernel   host    lo              local
127.255.255.255                 127.0.0.1       kernel   link    lo              local
200.1.1.3                       200.1.1.3       kernel   host    eth0            local
200.1.1.255                     200.1.1.3       kernel   link    eth0            local
root@pc3:/#
```

  On remarque que le routeur est r2 (200.1.1.1) qui est bien le routeur présent sur le réseau C.

3. Sur r1 et r2, j'ai effectué la commande `routel` pour vérifier la table de routage et trouver les LAN A, B, C et r1.

- Sur r1:

```
root@r1:/# routel
Dst             Gateway         Prefsrc         Protocol Scope   Dev             Table
100.0.0.8/30                    100.0.0.9       kernel   link    eth1
195.11.14.0/24                  195.11.14.1     kernel   link    eth0
200.1.1.0/24    100.0.0.10                                       eth1
100.0.0.9                       100.0.0.9       kernel   host    eth1            local
100.0.0.11                      100.0.0.9       kernel   link    eth1            local
127.0.0.0/8                     127.0.0.1       kernel   host    lo              local
127.0.0.1                       127.0.0.1       kernel   host    lo              local
127.255.255.255                 127.0.0.1       kernel   link    lo              local
195.11.14.1                     195.11.14.1     kernel   host    eth0            local
195.11.14.255                   195.11.14.1     kernel   link    eth0            local
root@r1:/#
```

  On remarque que le réseau A est bien présent ainsi que le réseau B. Et qu'on voit le routeur r2 (100.0.0.10).

- Sur r2:

```
root@r2:/# routel
Dst             Gateway         Prefsrc         Protocol Scope   Dev             Table
100.0.0.8/30                    100.0.0.10      kernel   link    eth1
195.11.14.0/24  100.0.0.9                                        eth1
200.1.1.0/24                    200.1.1.1       kernel   link    eth0
100.0.0.10                      100.0.0.10      kernel   host    eth1            local
100.0.0.11                      100.0.0.10      kernel   link    eth1            local
127.0.0.0/8                     127.0.0.1       kernel   host    lo              local
127.0.0.1                       127.0.0.1       kernel   host    lo              local
127.255.255.255                 127.0.0.1       kernel   link    lo              local
200.1.1.1                       200.1.1.1       kernel   host    eth0            local
200.1.1.255                     200.1.1.1       kernel   link    eth0            local
root@r2:/#
```

  On remarque que le réseau C est bien présent ainsi que le réseau B. Et qu'on voit le routeur r1 (100.0.0.9).

4. Sur pc3, on observe qu'initalement le cache ARP est vide avec la commande `arp -n` puis après un ping vers pc2, il contient l'adresse MAC de pc2.

```
root@pc3:/# arp -n
root@pc3:/# ping 200.1.1.7
PING 200.1.1.7 (200.1.1.7) 56(84) bytes of data.
64 bytes from 200.1.1.7: icmp_seq=1 ttl=64 time=5.11 ms
64 bytes from 200.1.1.7: icmp_seq=2 ttl=64 time=1.62 ms
a64 bytes from 200.1.1.7: icmp_seq=3 ttl=64 time=1.04 ms
rp 64 bytes from 200.1.1.7: icmp_seq=4 ttl=64 time=1.15 ms
-n64 bytes from 200.1.1.7: icmp_seq=5 ttl=64 time=1.27 ms
64 bytes from 200.1.1.7: icmp_seq=6 ttl=64 time=1.25 ms
64 bytes from 200.1.1.7: icmp_seq=7 ttl=64 time=0.996 ms
64 bytes from 200.1.1.7: icmp_seq=8 ttl=64 time=0.997 ms
64 bytes from 200.1.1.7: icmp_seq=9 ttl=64 time=0.894 ms
64 bytes from 200.1.1.7: icmp_seq=10 ttl=64 time=0.691 ms
^C
--- 200.1.1.7 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9012ms
rtt min/avg/max/mdev = 0.691/1.501/5.114/1.227 ms
root@pc3:/# arp -n
Address                  HWtype  HWaddress           Flags Mask            Iface
200.1.1.7                ether   00:00:00:00:00:02   C                     eth0
```

Sur Wireshark, on observe les requêtes ARP et les réponses ARP ainsi que les paquets ICMP échangés.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	00:00:00_00:00:03	Broadcast	ARP	60	Who has 200.1.1.7? Tell 200.1.1.3
2	0.001282866	00:00:00_00:00:02	00:00:00_00:00:03	ARP	60	200.1.1.7 is at 00:00:00:00:00:02
3	0.001542353	200.1.1.3	200.1.1.7	ICMP	98	Echo (ping) request id=0x0002, seq=1/256, ttl=64 (reply in 4)
4	0.002771023	200.1.1.7	200.1.1.3	ICMP	98	Echo (ping) reply id=0x0002, seq=1/256, ttl=64 (request in 3)
5	1.000776606	200.1.1.3	200.1.1.7	ICMP	98	Echo (ping) request id=0x0002, seq=2/512, ttl=64 (reply in 6)
6	1.001469550	200.1.1.7	200.1.1.3	ICMP	98	Echo (ping) reply id=0x0002, seq=2/512, ttl=64 (request in 5)
7	2.001716332	200.1.1.3	200.1.1.7	ICMP	98	Echo (ping) request id=0x0002, seq=3/768, ttl=64 (reply in 8)
8	2.002114119	200.1.1.7	200.1.1.3	ICMP	98	Echo (ping) reply id=0x0002, seq=3/768, ttl=64 (request in 7)
9	3.003318612	200.1.1.3	200.1.1.7	ICMP	98	Echo (ping) request id=0x0002, seq=4/1024, ttl=64 (reply in 10)
10	3.003575554	200.1.1.7	200.1.1.3	ICMP	98	Echo (ping) reply id=0x0002, seq=4/1024, ttl=64 (request in 9)
11	4.004959068	200.1.1.3	200.1.1.7	ICMP	98	Echo (ping) request id=0x0002, seq=5/1280, ttl=64 (reply in 12)
12	4.005502256	200.1.1.7	200.1.1.3	ICMP	98	Echo (ping) reply id=0x0002, seq=5/1280, ttl=64 (request in 11)
13	5.007292293	200.1.1.3	200.1.1.7	ICMP	98	Echo (ping) request id=0x0002, seq=6/1536, ttl=64 (reply in 14)
14	5.007656282	200.1.1.7	200.1.1.3	ICMP	98	Echo (ping) reply id=0x0002, seq=6/1536, ttl=64 (request in 13)
15	5.082262127	00:00:00_00:00:02	00:00:00_00:00:03	ARP	60	Who has 200.1.1.3? Tell 200.1.1.7
16	5.082528075	00:00:00_00:00:03	00:00:00_00:00:02	ARP	60	200.1.1.3 is at 00:00:00:00:00:03
17	6.008710544	200.1.1.3	200.1.1.7	ICMP	98	Echo (ping) request id=0x0002, seq=7/1792, ttl=64 (reply in 18)
18	6.009161483	200.1.1.7	200.1.1.3	ICMP	98	Echo (ping) reply id=0x0002, seq=7/1792, ttl=64 (request in 17)
19	7.010227989	200.1.1.3	200.1.1.7	ICMP	98	Echo (ping) request id=0x0002, seq=8/2048, ttl=64 (reply in 20)
20	7.010537071	200.1.1.7	200.1.1.3	ICMP	98	Echo (ping) reply id=0x0002, seq=8/2048, ttl=64 (request in 19)
21	8.010819308	200.1.1.3	200.1.1.7	ICMP	98	Echo (ping) request id=0x0002, seq=9/2304, ttl=64 (reply in 22)
22	8.011137287	200.1.1.7	200.1.1.3	ICMP	98	Echo (ping) reply id=0x0002, seq=9/2304, ttl=64 (request in 21)
23	9.011289251	200.1.1.3	200.1.1.7	ICMP	98	Echo (ping) request id=0x0002, seq=10/2560, ttl=64 (reply in 24)
24	9.011557153	200.1.1.7	200.1.1.3	ICMP	98	Echo (ping) reply id=0x0002, seq=10/2560, ttl=64 (request in 23)

On remarque que le trafic dans le même réseau est direct, il ne passe pas par le routeur r2. Les requêtes ARP sont en jaune et demandent l'adresse MAC de pc2. pc2 a aussi ajouté l'adresse MAC de pc3 dans son cache ARP.

```
root@pc2:/# arp -n
Address                  HWtype  HWaddress           Flags Mask            Iface
200.1.1.3                ether   00:00:00:00:00:03   C                     eth0
```

5. On remarque que le cache ARP de pc2 est initialement vide et qu'après le ping, il contient l'adresse MAC de r2 et non pas celle de pc1, car le ping passe par r2, le routeur du réseau C.

```
root@pc2:/# arp -n
Address                  HWtype  HWaddress           Flags Mask            Iface
200.1.1.3                ether   00:00:00:00:00:03   C                     eth0
root@pc2:/# ping 195.11.14.5
PING 195.11.14.5 (195.11.14.5) 56(84) bytes of data.
64 bytes from 195.11.14.5: icmp_seq=1 ttl=62 time=7.85 ms
64 bytes from 195.11.14.5: icmp_seq=2 ttl=62 time=2.27 ms
64 bytes from 195.11.14.5: icmp_seq=3 ttl=62 time=2.19 ms
64 bytes from 195.11.14.5: icmp_seq=4 ttl=62 time=2.66 ms
64 bytes from 195.11.14.5: icmp_seq=5 ttl=62 time=2.14 ms
64 bytes from 195.11.14.5: icmp_seq=6 ttl=62 time=2.59 ms
^C
--- 195.11.14.5 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5007ms
rtt min/avg/max/mdev = 2.136/3.282/7.848/2.051 ms
root@pc2:/# arp -n
Address                  HWtype  HWaddress           Flags Mask            Iface
200.1.1.3                ether   00:00:00:00:00:03   C                     eth0
200.1.1.1                ether   00:00:00:00:00:c1   C                     eth0
root@pc2:/#
```

Symétriquement, le cache ARP de pc1 est initialement vide et qu'après le ping, il contient l'adresse MAC de r1, routeur du réseau A.

```
root@pc1:/# arp -n
Address                  HWtype  HWaddress           Flags Mask            Iface
195.11.14.1              ether   00:00:00:00:00:a1   C                     eth0
```

Sur Wireshark, on observe les requêtes ARP et les réponses ARP ainsi que les paquets ICMP échangés.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	00:00:00_00:00:b2	Broadcast	ARP	60	Who has 100.0.0.9? Tell 100.0.0.10
2	0.000648772	00:00:00_00:00:b1	00:00:00_00:00:b2	ARP	60	100.0.0.9 is at 00:00:00:00:00:b1
3	0.000831920	200.1.1.7	195.11.14.5	ICMP	98	Echo (ping) request id=0x0003, seq=1/256, ttl=63 (reply in 4)
4	0.005063532	195.11.14.5	200.1.1.7	ICMP	98	Echo (ping) reply id=0x0003, seq=1/256, ttl=63 (request in 3)
5	0.999523100	200.1.1.7	195.11.14.5	ICMP	98	Echo (ping) request id=0x0003, seq=2/512, ttl=63 (reply in 6)
6	1.000474389	195.11.14.5	200.1.1.7	ICMP	98	Echo (ping) reply id=0x0003, seq=2/512, ttl=63 (request in 5)
7	2.001220554	200.1.1.7	195.11.14.5	ICMP	98	Echo (ping) request id=0x0003, seq=3/768, ttl=63 (reply in 8)
8	2.002065781	195.11.14.5	200.1.1.7	ICMP	98	Echo (ping) reply id=0x0003, seq=3/768, ttl=63 (request in 7)
9	3.003244640	200.1.1.7	195.11.14.5	ICMP	98	Echo (ping) request id=0x0003, seq=4/1024, ttl=63 (reply in 10)
10	3.004089140	195.11.14.5	200.1.1.7	ICMP	98	Echo (ping) reply id=0x0003, seq=4/1024, ttl=63 (request in 9)
11	4.004620772	200.1.1.7	195.11.14.5	ICMP	98	Echo (ping) request id=0x0003, seq=5/1280, ttl=63 (reply in 12)
12	4.005483178	195.11.14.5	200.1.1.7	ICMP	98	Echo (ping) reply id=0x0003, seq=5/1280, ttl=63 (request in 11)
13	5.006922385	200.1.1.7	195.11.14.5	ICMP	98	Echo (ping) request id=0x0003, seq=6/1536, ttl=63 (reply in 14)
14	5.007792226	195.11.14.5	200.1.1.7	ICMP	98	Echo (ping) reply id=0x0003, seq=6/1536, ttl=63 (request in 13)
15	5.063801600	00:00:00_00:00:b1	00:00:00_00:00:b2	ARP	60	Who has 100.0.0.10? Tell 100.0.0.9
16	5.064116280	00:00:00_00:00:b2	00:00:00_00:00:b1	ARP	60	100.0.0.10 is at 00:00:00:00:00:b2

On remarque que pc2 envoie une requête ARP pour obtenir l'adresse MAC de r2, puis envoie le paquet ICMP à r2 qui le transmet à r1, qui le transmet à pc1. pc1 répond directement à pc2 avec un paquet ICMP.

Au niveau des routeurs, on peut aussi remarquer que r2 a ajouté l'adresse MAC de pc2 dans son cache ARP:

```
root@r2:/# arp -n
root@r2:/# arp -n
Address                  HWtype  HWaddress           Flags Mask            Iface
100.0.0.9                ether   00:00:00:00:00:b1   C                     eth1
200.1.1.7                ether   00:00:00:00:00:02   C                     eth0
```

Et que r1 a ajouté l'adresse MAC de pc1 dans son cache ARP:

```
root@r1:/# arp -n
root@r1:/# arp -n
Address                  HWtype  HWaddress           Flags Mask            Iface
100.0.0.10               ether   00:00:00:00:00:b2   C                     eth1
195.11.14.5              ether   00:00:00:00:00:01   C                     eth0
```

6. Sur pc2, j'ai effectué la commande `traceroute` pour vérifier le chemin emprunté par les paquets vers pc1.

```
root@pc2:/# traceroute 195.11.14.5 -z 1
traceroute to 195.11.14.5 (195.11.14.5), 30 hops max, 60 byte packets
 1  200.1.1.1 (200.1.1.1)  15.330 ms  1.269 ms  0.549 ms
 2  100.0.0.9 (100.0.0.9)  51.149 ms  1.571 ms  1.974 ms
 3  195.11.14.5 (195.11.14.5)  10.888 ms  3.859 ms  2.205 ms
root@pc2:/#
```

On remarque que le paquet passe par r2 (200.1.1.1) puis par r1 (195.11.14.1) avant d'atteindre pc1 (195.11.14.2).

Sur Wireshark, on observe les paquets UDP envoyés par pc2 vers pc1 avec des valeurs TTL croissantes, les messages ICMP Time-to-live exceeded envoyés par r2 et r1 à pc2. On observe en jaune les requêtes ARP unicast émises par r2 et r1 durant le dialogue.

1	0.000000000	200.1.1.7	195.11.14.5	UDP	74	50400 → 33434 Len=32
2	0.007164180	200.1.1.1	200.1.1.7	ICMP	102	Time-to-live exceeded (Time to live exceeded in transit)
3	1.000056278	200.1.1.7	195.11.14.5	UDP	74	49924 → 33435 Len=32
4	1.000364267	200.1.1.1	200.1.1.7	ICMP	102	Time-to-live exceeded (Time to live exceeded in transit)
5	2.000172474	200.1.1.7	195.11.14.5	UDP	74	54094 → 33436 Len=32
6	2.000609975	200.1.1.1	200.1.1.7	ICMP	102	Time-to-live exceeded (Time to live exceeded in transit)
7	3.000405854	200.1.1.7	195.11.14.5	UDP	74	41408 → 33437 Len=32
8	3.002253106	100.0.0.9	200.1.1.7	ICMP	102	Time-to-live exceeded (Time to live exceeded in transit)
9	4.007741308	200.1.1.7	195.11.14.5	UDP	74	37867 → 33438 Len=32
10	4.008723478	100.0.0.9	200.1.1.7	ICMP	102	Time-to-live exceeded (Time to live exceeded in transit)
11	5.008060676	200.1.1.7	195.11.14.5	UDP	74	59763 → 33439 Len=32
12	5.009049899	100.0.0.9	200.1.1.7	ICMP	102	Time-to-live exceeded (Time to live exceeded in transit)
13	5.052314225	00:00:00_00:00:02	00:00:00_00:00:c1	ARP	60	Who has 200.1.1.1? Tell 200.1.1.7
14	5.052725504	00:00:00_00:00:c1	00:00:00_00:00:02	ARP	60	200.1.1.1 is at 00:00:00:00:00:c1
15	5.056055691	00:00:00_00:00:c1	00:00:00_00:00:02	ARP	60	Who has 200.1.1.7? Tell 200.1.1.1
16	5.056373859	00:00:00_00:00:02	00:00:00_00:00:c1	ARP	60	200.1.1.7 is at 00:00:00:00:00:02
17	6.007925859	200.1.1.7	195.11.14.5	UDP	74	35073 → 33440 Len=32
18	6.009542553	195.11.14.5	200.1.1.7	ICMP	102	Destination unreachable (Port unreachable)
19	7.008281245	200.1.1.7	195.11.14.5	UDP	74	34002 → 33441 Len=32
20	7.009522018	195.11.14.5	200.1.1.7	ICMP	102	Destination unreachable (Port unreachable)
21	8.009260866	200.1.1.7	195.11.14.5	UDP	74	38925 → 33442 Len=32
22	8.010838785	195.11.14.5	200.1.1.7	ICMP	102	Destination unreachable (Port unreachable)

On remarque aussi que le paquet UDP final envoyé par pc2 à pc1 a TTL=3.

21	8.009260866	200.1.1.7	195.11.14.5	UDP	74	38925 → 33442 Len=32
22	8.010838785	195.11.14.5	200.1.1.7	ICMP	102	Destination unreachable (Port unreachable)

```
▸ Frame 21: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface eth1, id 0
▾ Ethernet II, Src: 00:00:00_00:00:02 (00:00:00:00:00:02), Dst: 00:00:00_00:00:c1 (00:00:00:00:00:c1)
  ▾ Destination: 00:00:00_00:00:c1 (00:00:00:00:00:c1)
      .... ..0. .... .... .... .... = LG bit: Globally unique address (factory default)
      .... ...0 .... .... .... .... = IG bit: Individual address (unicast)
  ▾ Source: 00:00:00_00:00:02 (00:00:00:00:00:02)
      .... ..0. .... .... .... .... = LG bit: Globally unique address (factory default)
      .... ...0 .... .... .... .... = IG bit: Individual address (unicast)
    Type: IPv4 (0x0800)
    [Stream index: 0]
▾ Internet Protocol Version 4, Src: 200.1.1.7, Dst: 195.11.14.5
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  ▸ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 60
    Identification: 0xee5a (61018)
  ▸ 000. .... = Flags: 0x0
    ...0 0000 0000 0000 = Fragment Offset: 0
  ▸ Time to Live: 3
    Protocol: UDP (17)
    Header Checksum: 0x2f3e [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 200.1.1.7
    Destination Address: 195.11.14.5
    [Stream index: 0]
▸ User Datagram Protocol, Src Port: 38925, Dst Port: 33442
▸ Data (32 bytes)
```

7. J'ai séparé les deux cas pour plus de clarté.

   - Pour illustrer les messages d'erreur et les paquets dans le domaine de collision local, sur pc1, j'ai pingé une adresse locale non attribuée (195.11.14.4) avec la commande `ping 195.11.14.4`.
     Le message d'erreur est "Destination Host Unreachable":

```
root@pc1:/# ping 195.11.14.4
PING 195.11.14.4 (195.11.14.4) 56(84) bytes of data.
From 195.11.14.5 icmp_seq=1 Destination Host Unreachable
From 195.11.14.5 icmp_seq=2 Destination Host Unreachable
From 195.11.14.5 icmp_seq=3 Destination Host Unreachable
^C
--- 195.11.14.4 ping statistics ---
5 packets transmitted, 0 received, +3 errors, 100% packet loss, time 4094ms
pipe 3
root@pc1:/#
```

On observe aussi sur Wireshark (connecté au domaine de collision A) des requêtes ARP Broadcast pour obtenir l'adresse MAC de l'adresse IP recherchée (on ne passe pas par le routeur car l'adresse est locale).

1	0.000000000	00:00:00_00:00:01	Broadcast	ARP	60	Who has 195.11.14.4? Tell 195.11.14.5
2	1.022584226	00:00:00_00:00:01	Broadcast	ARP	60	Who has 195.11.14.4? Tell 195.11.14.5
3	2.046967443	00:00:00_00:00:01	Broadcast	ARP	60	Who has 195.11.14.4? Tell 195.11.14.5
4	3.070939231	00:00:00_00:00:01	Broadcast	ARP	60	Who has 195.11.14.4? Tell 195.11.14.5
5	4.094154140	00:00:00_00:00:01	Broadcast	ARP	60	Who has 195.11.14.4? Tell 195.11.14.5
6	5.118250580	00:00:00_00:00:01	Broadcast	ARP	60	Who has 195.11.14.4? Tell 195.11.14.5

- Pour illustrer les messages d'erreur et les paquets dans le domaine de collision non local, sur pc1, j'ai pingé une adresse non locale non attribuée (200.1.1.5) avec la commande `ping 200.1.1.5`. Le message d'erreur est "Destination Host Unreachable":

```
root@pc1:/# ping 200.1.1.5
PING 200.1.1.5 (200.1.1.5) 56(84) bytes of data.
From 100.0.0.10 icmp_seq=1 Destination Host Unreachable
From 100.0.0.10 icmp_seq=2 Destination Host Unreachable
From 100.0.0.10 icmp_seq=3 Destination Host Unreachable
From 100.0.0.10 icmp_seq=4 Destination Host Unreachable
From 100.0.0.10 icmp_seq=5 Destination Host Unreachable
From 100.0.0.10 icmp_seq=6 Destination Host Unreachable
From 100.0.0.10 icmp_seq=7 Destination Host Unreachable
From 100.0.0.10 icmp_seq=8 Destination Host Unreachable
From 100.0.0.10 icmp_seq=9 Destination Host Unreachable
From 100.0.0.10 icmp_seq=10 Destination Host Unreachable
From 100.0.0.10 icmp_seq=11 Destination Host Unreachable
From 100.0.0.10 icmp_seq=12 Destination Host Unreachable
From 100.0.0.10 icmp_seq=13 Destination Host Unreachable
From 100.0.0.10 icmp_seq=14 Destination Host Unreachable
^C
--- 200.1.1.5 ping statistics ---
14 packets transmitted, 0 received, +14 errors, 100% packet loss, time 13217ms
pipe 4
root@pc1:/#
```

On observe aussi sur Wireshark (connecté au domaine de collision C) une requête ARP pour obtenir l'adresse MAC associée à l'adresse IP recherchée (on passe par le routeur car l'adresse n'est pas locale et donc la requête est adressée au nom du routeur r2).

1	0.000000000	00:00:00_00:00:c1	Broadcast	ARP	60	Who has 200.1.1.5? Tell 200.1.1.1
2	1.024873787	00:00:00_00:00:c1	Broadcast	ARP	60	Who has 200.1.1.5? Tell 200.1.1.1
3	2.049161281	00:00:00_00:00:c1	Broadcast	ARP	60	Who has 200.1.1.5? Tell 200.1.1.1
4	3.074074990	00:00:00_00:00:c1	Broadcast	ARP	60	Who has 200.1.1.5? Tell 200.1.1.1
5	4.097054238	00:00:00_00:00:c1	Broadcast	ARP	60	Who has 200.1.1.5? Tell 200.1.1.1
6	5.121199691	00:00:00_00:00:c1	Broadcast	ARP	60	Who has 200.1.1.5? Tell 200.1.1.1
7	7.114860959	00:00:00_00:00:c1	Broadcast	ARP	60	Who has 200.1.1.5? Tell 200.1.1.1
8	8.127390554	00:00:00_00:00:c1	Broadcast	ARP	60	Who has 200.1.1.5? Tell 200.1.1.1
9	9.151032012	00:00:00_00:00:c1	Broadcast	ARP	60	Who has 200.1.1.5? Tell 200.1.1.1
10	10.176439773	00:00:00 00:00:c1	Broadcast	ARP	60	Who has 200.1.1.5? Tell 200.1.1.1

Avec Wireshark connecté au domaine de collision A, on observe une série de requêtes ARP envoyées par pc1 pour tenter de résoudre l'adresse MAC de la destination (195.11.14.5x), suivies de paquets ICMP "Destination unreachable (Host unreachable)" générés par le routeur r1. Cela montre que le routeur tente d'acheminer le paquet, mais ne trouve pas de correspondance ARP pour la destination, et informe donc l'émetteur que l'hôte est injoignable. Les requêtes ARP

restent sans réponse, ce qui confirme que l'adresse IP cible n'est pas attribuée sur le réseau.

1	0.000000000	195.11.14.5	200.1.1.5	ICMP	98	Echo (ping) request id=0x0008, seq=1/256, ttl=64 (no response found!)
2	1.025655097	195.11.14.5	200.1.1.5	ICMP	98	Echo (ping) request id=0x0008, seq=2/512, ttl=64 (no response found!)
3	2.049621128	195.11.14.5	200.1.1.5	ICMP	98	Echo (ping) request id=0x0008, seq=3/768, ttl=64 (no response found!)
4	3.073528305	195.11.14.5	200.1.1.5	ICMP	98	Echo (ping) request id=0x0008, seq=4/1024, ttl=64 (no response found!)
5	3.074235492	100.0.0.10	195.11.14.5	ICMP	126	Destination unreachable (Host unreachable)
6	3.074535341	100.0.0.10	195.11.14.5	ICMP	126	Destination unreachable (Host unreachable)
7	3.074762521	100.0.0.10	195.11.14.5	ICMP	126	Destination unreachable (Host unreachable)
8	4.075573639	195.11.14.5	200.1.1.5	ICMP	98	Echo (ping) request id=0x0008, seq=5/1280, ttl=64 (no response found!)
9	5.089284609	195.11.14.5	200.1.1.5	ICMP	98	Echo (ping) request id=0x0008, seq=6/1536, ttl=64 (no response found!)
10	5.153962404	00:00:00_00:00:01	00:00:00_00:00:a1	ARP	60	Who has 195.11.14.1? Tell 195.11.14.5
11	5.154273786	00:00:00_00:00:a1	00:00:00_00:00:01	ARP	60	195.11.14.1 is at 00:00:00:00:00:a1
12	6.114191425	195.11.14.5	200.1.1.5	ICMP	98	Echo (ping) request id=0x0008, seq=7/1792, ttl=64 (no response found!)
13	6.145810641	100.0.0.10	195.11.14.5	ICMP	126	Destination unreachable (Host unreachable)
14	6.146062948	100.0.0.10	195.11.14.5	ICMP	126	Destination unreachable (Host unreachable)
15	6.146297432	100.0.0.10	195.11.14.5	ICMP	126	Destination unreachable (Host unreachable)
16	6.146616548	100.0.0.10	195.11.14.5	ICMP	126	Destination unreachable (Host unreachable)
17	7.115396741	195.11.14.5	200.1.1.5	ICMP	98	Echo (ping) request id=0x0008, seq=8/2048, ttl=64 (no response found!)
18	8.225398309	00:00:00_00:00:a1	00:00:00_00:00:01	ARP	60	Who has 195.11.14.5? Tell 195.11.14.1
19	8.225651268	00:00:00_00:00:01	00:00:00_00:00:a1	ARP	60	195.11.14.5 is at 00:00:00:00:00:01
20	10.175300240	100.0.0.10	195.11.14.5	ICMP	126	Destination unreachable (Host unreachable)

Conclusion

Ce premier TP m'a permis de me familiariser avec l'outil Kathará et de revoir les notions de base du réseau, notamment le fonctionnement des adresses IP, des tables de routage, du protocole ARP et des messages ICMP. J'ai pu observer concrètement comment les paquets sont acheminés dans un réseau avec plusieurs sous-réseaux et routeurs, ainsi que les mécanismes de résolution d'adresse et de gestion des erreurs. L'utilisation de Wireshark pour capturer et analyser le trafic réseau a été intéréssante de par sa simplicité mais aussi car j'ai pu visualiser en temps réel les échanges sur les différents réseaux.